

HealthPro · Моє Здоров'я

Фаза 2 + Етап 5 — Capacitor APK та локальна БД (SQLite)

28 квітня 2026 р.

1 · Резюме

Цей звіт фіксує завершення Фази 2 (нативний Android APK через Capacitor) та Етапу 5 (локальна БД SQLite на нативі). Поточний раунд закрив три фінальні баги PrePhase2, налаштував збірку APK через GitHub Actions, додав 8 Capacitor-плагінів, інтегрував @capacitor-community/sqlite та переробив шар persistence на трирівневу архітектуру (SQLite → IndexedDB → localStorage-mirror).

ФАЗА 2 ЧАСТИНА 1: 3 БАГИ ЗАКРИТО

ФАЗА 2 ЧАСТИНА 2: APK ЗІБРАНО

ЕТАП 5: SQLite ІНТЕГРОВАНО

ТЕСТИ: 41/41 ЗЕЛЕНІ

1 ЗАЛИШКОВИЙ БАГ ВИЯВЛЕНО НА ПРИСТРОЇ

2 · Виправлені баги PrePhase2

#	Бог	Корінь причини	Виправлення
1	Sticky-навігація переміщувалася при зміні екрану	Жорстке top:62px у layout.css	CSS-змінна --header-height + JS updateHeaderHeight()
2	Модалка експорту не закривалася після натискання	Дії викликалися напряму без post-action логіки	Обгортки exportPDF / printReportPeriod /
3	Кнопка CSV зовні не реагувала	Дія викликала exportReportCSV() без обгортки	Та сама обгортка проксує getExportMeasurements у

3 · Збірка Android APK через GitHub Actions

Replit-середовище не має Java та Android SDK (~5 ГБ зайняв би SDK). Тому збірка винесена на GitHub Actions.

Workflow [.github/workflows/android-apk.yml](#)

- Тригери: push до main, pull_request, workflow_dispatch (ручний запуск).
- JDK 21 (Temurin), Android SDK через android-actions/setup-android.
- npm ci у root (для cap sync) + npm ci у HealthPro-Moie-Zdorovia/ (для імпортів).
- Vite build → npx cap sync android → ./gradlew assembleDebug.
- APK перейменовується у HealthPro-debug-<git-sha>.apk і вантажиться як артефакт HealthPro-debug-apk (зберігається 30 днів).

Як отримати APK

GitHub → репо → вкладка Actions → workflow «Build Android APK» → останній run → блок Artifacts низу → завантажити HealthPro-debug-apk.zip → всередині HealthPro-debug-<sha>.apk.

4 · Capacitor плагіни (9 шт.)

Плагін	Версія	Призначення
@capacitor/app	8.1.0	Lifecycle (foreground/background, deep link)
@capacitor/filesystem	8.1.2	Файлове сховище (для майбутнього експорту)
@capacitor/haptics	8.0.2	Тактильний відгук (натиск кнопок)
@capacitor/local-notifications	8.0.2	Нагадування про прийом ліків та вимірювання
@capacitor/preferences	8.0.1	KV-сховище (lang, theme, дрібні налаштування)
@capacitor/share	8.0.1	Системний share-sheet (для майбутнього експорту)
@capacitor/splash-screen	8.0.1	Splash екран при старті
@capacitor/status-bar	8.0.2	Стиль/колір статус-бара
@capacitor-community/sqlite	8.1.0	Реляційна БД (Етап 5)

Усі плагіни встановлено в обидва package.json (root для cap sync поряд з capacitor.config.json, HealthPro-Moie-Zdorovia для ESM-імпортів у бандлі). cap sync android підтверджує: Found 9 Capacitor plugins for android.

5 · Етап 5 — Локальна БД SQLite

Архітектура тривіневого persistence

Жоден feature-модуль не змінений. Зміна локалізована у src/core/storage.js + новий src/core/sqlite.js.

Рівень	Web / PWA	Native (Android)
1. Sync mirror	localStorage	localStorage
2. Async primary	IndexedDB (HealthProDB / state)	SQLite (HealthProDB.db, kv_state)
3. Backup	—	IndexedDB (як другий запис)

Схема SQLite

```
CREATE TABLE IF NOT EXISTS kv_state (  
  k      TEXT PRIMARY KEY,  
  v      TEXT NOT NULL,      -- JSON-stringified  
  updated_at INTEGER NOT NULL  -- ms since epoch  
);
```

Чому KV-таблиця, а не реляційна схема: feature-модулі і далі читають/пишуть масиви та об'єкти; міграція з IndexedDB робиться 1:1 копіюванням (key, JSON.stringify(value)); реляційні таблиці (наприклад окрема measurements з індексами по даті) — це майбутня v2-міграція без зламу API.

Чому SQLite надійніше за IndexedDB на Android

- SQLite файл лежить у /data/data/<package>/databases/HealthProDB.db — приватне сховище додатка.
- Не залежить від кешу WebView; «Очистити кеш» в системних налаштуваннях не зачіпає БД.
- Дані пропадають тільки при «Стерти дані» / видаленні додатка — як у нативних застосунків.

Міграційна логіка bootstrapStorage()

- 1. Запит persistent storage permission.
- 2. Ініціалізація SQLite (no-op на вебі).
- 3. Одноразова міграція legacy localStorage → primary (для старих ключів measurements/pills/...).
- 4. Одноразова міграція IndexedDB → SQLite (тільки на нативі, тільки якщо SQLite порожня).
- 5. Регідрат localStorage-mirror з primary, щоб наступний холодний старт малював актуальні дані синхронно.

6 • Залишковий баг для наступного раунду

Баг 4 (виявлено на пристрої): експорт PDF / CSV не зберігає файл

Симптом: модалка показує ім'я файлу «healthpro-...-...csv» / «...-pdf», але фізично файл не з'являється у файловому менеджері Android.

Корінь причини

- csv.js, json-backup та platform.js::download() використовують web-механізм <a download>.click().
- jsPDF.save() усередині pdf.js робить те саме під капотом.
- У Android WebView такий download або тихо ігнорується, або кладе файл у непублічний WebView-cache.

План виправлення (наступний раунд)

- Розділити шлях у platform.js::download(): якщо isNative() → Filesystem.writeFile у Directory.Documents → потім Share.share для системного «Зберегти / Поділитися».
- Переробити csv.js: замість document.createElement('a') → виклик platform.download().
- Переробити pdf.js: замість doc.save(...) → doc.output('blob') → platform.download(blob).
- Додати unit-тест з мок-об'єктом window.Capacitor.Plugins.{Filesystem,Share}.
- Перезібрати APK і повторно тестувати на пристрої.

7 • Метрики

Параметр	Значення
Тести (vitest)	41 / 41 зелені
Тривалість тестів	~1.2s
Vite build	успіх, ~5.3s
Розмір основного бандла	1000 КБ (gzip 448 КБ)
Розмір CSS	40 КБ (gzip 8.2 КБ)
Capacitor-плагіни	9 (на Android)
Розмір SQLite (порожня БД)	~12 КБ файл
Файлів змінено цього раунду	4 (storage.js, sqlite.js, package.json ×2)

8 • Структура (ключові файли)

```
repo-root/  
  capacitor.config.json      # appId ua.healthpro.app, webDir: dist
```

```

package.json      # cap CLI + усі плагіни (для cap sync)
android/          # нативний проект (не редагуємо)
dist/             # build output (не в git)
.github/workflows/
  android-apk.yml  # збірка APK
  ci.yml           # тести + build на push
scripts/
  generate_phase2_stage5_report.cjs # цей звіт
HealthPro-Moie-Zdorovia/
  package.json     # web-залежності + ті ж плагіни
  src/core/
    storage.js     # 3-tier persistence (новий)
    sqlite.js      # SQLite адаптер (новий, по-ор на вебi)
    platform.js    # детекція native + plugin проху
    state.js       # глобальний стан
  src/features/export/
    index.js       # обгортки з auto-close модалки

```

9 • Статус та наступні кроки

ГОТОВО ДО ТЕСТУВАННЯ APK НА ПРИСТРОЇ

Поточна гілка main містить SQLite-інтеграцію. Після git push GitHub Actions зібере новий APK з усіма 9 плагінами.

Послідовність

- 1. git push origin main → GitHub Actions автоматично запускається.
- 2. Отримати APK з артефактів, встановити на пристрій.
- 3. Перевірити перенос даних: відкрити старий APK (де є дані з IDB), оновити на новий → дані мають з'явитися (міграція IDB → SQLite виконається автоматично при першому запуску).
- 4. Зробити кілька записів тиску, ліків — закрити додаток, очистити WebView-cache в системі → перевірити що дані лишилися (це і є вигода SQLite над IDB).
- 5. Підтвердити Баг 4 (експорт CSV/PDF файл не знаходиться) — після цього запускається наступний раунд правок.